

Learning Textual Entailment and Abduction

Gary Nan Tie, Aug 13, 2026

Abstract

We introduce textual entailment and abductive reasoning, in the context of a fast parallel version of Alice, a state machine LLM that combines a recurrent implicit deep learning model transition with an attention-based output over learned fixed points, optimized for syntactic prediction and semantic coherence within a word embedding space.

Let $\mathcal{E} = \{t_1, \dots, t_m\}$, token $t_i \in \mathbb{R}^p$

Pre-train $\text{Ecila}(\mathcal{S}, \mathcal{O}')$ on $\mathcal{E}$ to learn $\mathcal{S}$ (see Appendix)

ie $h_0 = \mathbf{0} \in \mathbb{R}^r$, $k_0 = \mathbf{0} \in \mathbb{R}^q$

$u_i = \begin{pmatrix} t_i \\ h_{i-1} \end{pmatrix} \in \mathbb{R}^r$, $r = p + q$, $i \in [m]$

$k_i = \varphi(Ak_i + Bu_i) \in \mathbb{R}^q$

$h_i = \mathcal{S}(h_{i-1}, t_i) \triangleq Ck_i + Du_i \in \mathbb{R}^r$

→

Suppose $\text{Alice}(\mathcal{S}, \mathcal{O})$ processes two token strings,

premise P and hypothesis Q , $P = (p_1, \dots, p_r)$, $Q = (q_1, \dots, q_s)$,

as follows: $h_j^P = \mathcal{S}(h_{j-1}^P, p_j)$, $j \in [r]$, $h_0^P = \mathbf{0}$

$h_j^Q = \mathcal{S}(h_{j-1}^Q, q_j)$, $j \in [s]$, $h_0^Q = \mathbf{0}$

To obtain $h_P \triangleq h_r^P$ and $h_Q \triangleq h_s^Q$

Let h'_i denote the final update in a stack of attention layers in output $\mathcal{O}(h_i)$ of $\text{Alice}(\mathcal{S}, \mathcal{O})$

(see Appendix). We drop the token index i unless necessary.

Define $\beta_j(h) = \text{softmax}_{j \in [m]} \text{score}(h', k_j)$

and let $\beta(h) = (\beta_1(h), \dots, \beta_m(h))$ be the distribution of h over $\{k_j\}_{j=1}^m$.

For $0 < \tau < 1$, define $F_\tau(h) = \{k_j \mid \beta_j(h) > \tau\}$

Define entailment^{*} $P \models_\tau Q$ iff $F_\tau(h_Q) \subseteq F_\tau(h_P)$.

Define contradiction $P \models_k \neg Q$

iff $|F(h_P) \cap F(h_Q)| < k$, an integer.

* transitive

On training triples (P, Q, y)

where $y \in \{\text{entail}_T, \text{neutral}, \text{contradiction}_K\}$

$$\text{learn Ent}(h_P, h_Q) = \begin{cases} \text{entail}_T & \text{if } P \models Q \\ \text{contradiction}_K & \text{if } P \models \neg Q \\ \text{neutral} & \text{otherwise} \end{cases}$$

to minimize loss $\mathcal{L} = \mathcal{L}_{\text{Perplexity}} + \lambda \mathcal{L}_{\text{Renyi}} + \rho \mathcal{L}_F$

where $\mathcal{L}_F$ is a loss function for Ent

and $\mathcal{L}_{\text{Perplexity}}$ and $\mathcal{L}_{\text{Renyi}}$ are loss functions

for Perplexity $(o_2, \dots, o_{m+1})$ and average Renyi α -entropy

$E(\{t_i\}_{i=1}^m)$ respectively (see Appendix).

Hence Alice $(\mathcal{S}, \mathcal{O})$ can learn entailment $\models$.

Abduction

Given a hypothesis Q , we wish to find a premise P such that $P \models Q$.

Token string Q may be autoregressively continued by $\text{Alice}(\mathcal{S}, Q)$ to generate $Q, P_1, P_2, \dots, P_n, n \geq 1$, where P_j denotes a continuation sentence.

Choose $0 < \tau < 1$, for each P_j check

if $P_j \models_{\tau} Q$. Suppose $\mathcal{P} = \{P_j \mid P_j \models_{\tau} Q\} \neq \emptyset$

let $m = \min_{P_j \in \mathcal{P}} |F_{\tau}(h_{P_j})|$ and

P_{j^*} be the first P_j with $|F_{\tau}(h_{P_j})| = m$.

Abduction: Given nonempty $\mathcal{P}$,

choose $P_{j^*} \in \mathcal{P}$ to entail Q ; $P_{j^*} \models Q$, is

a parsimonious selection.

Appendix

State Machine $(\mathcal{S}, \mathcal{O})$

Input: target tokens $t_1, \dots, t_m \in \mathbb{R}^p$

Initial state: $h_0 = \mathbf{0} \in \mathbb{R}^q$

Next state: $h_i = \mathcal{S}(h_{i-1}, t_i) \in \mathbb{R}^q, \quad i \in [m]$

Output: $O_{i+1} = \mathcal{O}(h_i) \in \mathbb{R}^p, \quad i \in [m]$

where $\mathcal{S}$ is a transition function and

$\mathcal{O}$ is an output function.

—

Let $\text{Emb} : \mathbb{R}^P \rightarrow \mathbb{R}^{P'}$ be a word embedding

$$t_i \mapsto v_i = \begin{bmatrix} v_i^1 \\ \vdots \\ v_i^B \end{bmatrix}$$

where v_i has been partitioned into B blocks.

Given input token string $t_1, \dots, t_m \in \mathbb{R}^P$

we will define a state machine $\text{Ecila}^j(\mathcal{S}^j, \mathcal{O}^j)$

with transition function $\mathcal{S}^j(h_{i-1}^j, t_i)$ and

output function $\mathcal{O}^j(h_i^j)$, where h_i^j is chosen

so that $\hat{v}_i^j \triangleq \mathcal{O}^j(h_i^j) \approx v_i^j$.

Define initial hidden state $h_0 = \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_B \end{bmatrix} \in \mathbb{R}^q$, $q = Bb$,

where independent draws $\epsilon_j \sim \text{Normal}_{\mathbb{R}^b}(0, 1)$

For $j \in [B]$, implicit RNN $\mathcal{S}^j$:

$$h_0^j \triangleq \varepsilon_j \in \mathbb{R}^b, \quad k_0^j = 0 \in \mathbb{R}^b$$

For $i \in [m]$, $u_i^j = \begin{bmatrix} t_i \\ h_{i-1}^j \end{bmatrix} \in \mathbb{R}^{p+b}$, let $r = p+b$

$$k_i^j \triangleq \varphi^j \left(\underset{b \times b}{A^j} k_i^j + \underset{b \times r}{B^j} u_i^j \right) \in \mathbb{R}^b$$

$$h_i^j \triangleq \underset{b \times b}{C^j} k_i^j + \underset{b \times r}{D^j} u_i^j \in \mathbb{R}^b$$

where $A^j = \Gamma^j - \text{diag}(|\Gamma^j| \mathbf{1}_b) + \gamma \text{Id}_b$, $0 < \gamma < 1$,

ensures A^j is well-posed for φ^j

Define transition function $\mathcal{S}^j(h_{i-1}^j, t_i) \triangleq h_i^j$

Let $\Theta_j' = \{(W_1, b_1), (W_2, b_2)\}$ denote parameters

for FeedForward network:

$$\text{FFN}_{\Theta_j'}(h_i^j) \triangleq \text{ReLU}(h_i^j W_1 + b_1) W_2 + b_2$$

Define output function $\hat{v}_i^j = o_i^j = \mathcal{O}^j(h_i^j) = \text{FFN}_{\Theta_j'}(h_i^j)$

This completes the definition of state machine $\text{Ecila}^j(\mathcal{S}^j, \mathcal{O}^j)$.

Recall the Chebyshev metric for n -vectors:

$$\text{Cheb}(a, b) = \max_{i \in [n]} (|a_i - b_i|)$$

Ecila^j($\mathcal{S}^j, \mathcal{O}^j$) training problem

Learn implicit RNN matrices A^j, B^j, C^j, D^j ,
and fixed points k_i^j , and feedforward network
parameters θ_j' to minimize:

$$\sum_{i=1}^m \text{Cheb}(o_i^j, v_i^j)$$

where $o_i^j = \mathcal{O}^j(h_i^j) = \text{FFN}_{\theta_j'}(h_i^j)$

and $h_i^j = C^j k_i^j + D^j u_i^j$.

$$k_i^j = \varphi^j(A^j k_i^j + B^j u_i^j)$$

$Ecila(\mathcal{S}, \mathcal{O}')$ is the parallel state machine

$Parallel(Ecila^1, \dots, Ecila^B)$

with $\mathcal{S} \triangleq \mathcal{S}^1 | \dots | \mathcal{S}^B$ and $\mathcal{O}' = \mathcal{O}^1 | \dots | \mathcal{O}^B$.

$Ecila(\mathcal{S}, \mathcal{O}')$ is learned in parallel by B state machines $Ecila^j(\mathcal{S}^j, \mathcal{O}^j)$ to simulate Emb.

From $Ecila(\mathcal{S}, \mathcal{O}')$ discard $\mathcal{O}'$ and retain $\{h_i\}_{i=1}^m$, $\{k_j\}_{j=1}^m$ and $\{v_j\}_{j=1}^m$ to define $\mathcal{O}$ for $Alice(\mathcal{S}, \mathcal{O})$.

NB: Let h'_i denote the final update in a stack of attention layers. As we are learning $\mathcal{S}$ and $\mathcal{O}$ in stages

here distribution $\beta_{ij} \triangleq \text{softmax}_{j \in [m]} \text{score}(h'_i, k_j)$

so instead of $j \in [i]$ for joint learning, $j \in [m]$

for learning in stages, since we know $\{(k_j, v_j)\}_{j=1}^m$.

Attention layer:

Input h_i .

Let score be a desired similarity function,

like a reproducing kernel.

For $i \in [m]$, define probabilities

$$\alpha_{ij} = \underset{j \in [i]}{\text{softmax}} \text{ score}(h_i, k_j)$$

Let $\hat{h}_i = \sum_{j=1}^i \alpha_{ij} v_j$ and define update

$$h'_i = \text{FFN}_{\theta}(\hat{h}_i) \quad (\dim h'_i = \dim k_j)$$

Output h'_i

—

Stack L attention layers $\Theta \triangleq \{\theta_1, \dots, \theta_L\}$

composing updates FFN_{θ_L} and now let

h'_i denote the final L -th update.

$h'(v)$ where $v = \text{Emb}(t)$

After L layers, for $i \in [m]$ define probabilities

$$\beta_{ij} = \text{softmax}_{j \in [J]} \text{score}(h_i^l, k_j) \text{ and}$$

$$\text{let } j_i^\beta = \arg \max_{j \in [J]} \beta_{ij}.$$

Define token continuation $\phi_{i+1} = \phi(h_i) \triangleq t_{j_i^\beta}$

So attention layers construct ϕ .

Define the average Renyi α -entropy of $\{t_1, \dots, t_m\}$ by:

$$E(\{t_i\}_{i=1}^m) \triangleq \frac{1}{m} \sum_{i=1}^m H_\alpha^i$$

where $H_\alpha^i \triangleq H_\alpha(\{\beta_{ij}\}_{j=1}^J)$, H_α Renyi α -entropy.

Token continuation $t_1, t_2, \dots, t_i, o_{i+1}$ via:

Alice $(S, \mathcal{O})$ training problem (not-token)

Let $\lambda > 0$, learn implicit RNN matrices A, B, C, D
and fixed points k_i , as well as, feedforward network
parameters Θ to minimize:

$$[\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\}_{i=1}^m)]$$

—

Next-token prediction

Initialization: let $t_1 = \text{empty token or a seed token}$

Given t_1 , define $t_2 = o_2$.

Given t_1, t_2 define $t_3 = o_3$

Given t_1, t_2, t_3 define $t_4 = o_4$ and so on, ...

—

NB: token continuation Θ_{i+1} is shaped

by perplexity and Renyi α -entropy.

Sequence-to-Sequence learning

target tokens $t_1, \dots, t_m$
Input: source tokens $\tilde{t}_1, \dots, \tilde{t}_m$

For $\{t_i\}$ and $\{\tilde{t}_j\}$ solve their respective

Alice next-token learning problem to find their
contextual representations $\{h'_i\}$ and $\{\tilde{h}'_j\}$

Define $S_{ij} = \underset{j \in [n]}{\text{softmax score}}(h'_i, \tilde{h}'_j)$, $i \in [m]$

and let $j_i^{\delta} = \underset{j \in [n]}{\arg \max} S_{ij}$.

Given target t_i , $i \in [m]$,

define source output $o_i \triangleq \tilde{t}_{j_i^{\delta}}$

So $t_1, \dots, t_m$ is translated to $o_1, \dots, o_m$

where $o_i \in \{\tilde{t}_j\}_{j=1}^n$.

—

The Alice in Parallel training problem:

Given $\{h_i\}_{i=1}^m$ and $\{(k_j, v_j)\}_{j=1}^m$ from $\text{Ecila}(\mathcal{S}, \mathcal{O}')$
and $\lambda > 0$, learn Feedforward network parameters

$\Theta = \{\theta_1, \dots, \theta_L\}$ for L attention layers to minimize:

$$\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\}_{i=1}^m)$$

where $t_1, \dots, t_m$ are the corpus tokens.

~~it~~

References

[1] 'Alice Through the Word Embedding'

Gary Nan Tie, Jul 2, 2026

DOI: 10.13140/RG.2.2.34901.79841

https://www.researchgate.net/publication/408460384_Alice_Through_the_Word_Embedding?channel=doi&linkId=6a4921a95ec65810298eeca&showFulltext=true

[2] 'Alice in Parallel'

Gary Nan Tie, Jul 18, 2026

DOI: 10.13140/RG.2.2.18755.36641/1

https://www.researchgate.net/publication/410626715_Alice_in_Parallel?channel=doi&linkId=6a64f3e2675a850148c9fa9a&showFulltext=true